Appendix: AI Assistant Usage Methodology

Smart Sensor Network System (SSNS)

Muhammad Saleem
M.Sc. High Integrity Systems
Frankfurt University of Applied Sciences
Frankfurt am Main, Germany
muhammad.saleem2@stud.fra-uas.de

APPENDIX: AI ASSISTANT USAGE METHODOLOGY

In accordance with the academic integrity policy, this appendix documents how AI coding assistants were used during the development of the Smart Sensor Network System (SSNS).

Two assistants were used at different stages of the project, for different purposes:

- **Google Antigravity (Gemini)** was used for early-stage technical-documentation consolidation (e.g. keeping `docs/final/` in sync with the codebase).
- **Claude Code (Anthropic)** was used for implementation-level work: root-cause debugging, quantitative evaluation of the ML pipeline, documentation-vs-implementation audits, and test-suite engineering.

The AI was used strictly as a **verification and implementation tool against an existing, working codebase** – not as a source of academic content. The report's introduction, analysis, discussion, and conclusions were written entirely by the author; none of the examples below involve report prose. Every prompt below was issued in response to a concrete engineering question (a bug, a discrepancy, a missing test, a documentation gap) that the author had already identified; the AI's role was to investigate the actual source code, confirm or refute the hypothesis, and implement or draft the fix.

The prompting process itself followed a consistent two-step pattern: the underlying engineering question was first drafted as a plain problem statement, then refined into a precise, unambiguous technical instruction (in some cases with the assistance of a general-purpose LLM used purely for this rephrasing step) before being issued to the coding assistant responsible for investigating the codebase. The prompts below reflect that refined form, since it is what was actually submitted. Routine debugging, syntax corrections, and Git management commands have been omitted for brevity. Prompts and responses below have been lightly edited for length; no technical claims have been altered.

This appendix spans the full development history of the project (initial commit through the current branch), organized into three phases: foundational architecture, coordinated multi-tier feature engineering, and the verification/debugging/test-engineering work that closed out the project.

Part I—Foundational Architecture

1. Store-and-Forward Reliability: From NVS to CBOR to Redis

User Prompt:

> The very first version of the sensor node just fires a CoAP packet and hopes for the best – if the gateway doesn't respond, the reading is gone. I want an actual store-and-forward guarantee: a CoAP ACK listener so the sensor knows a packet was really received, persistent local queueing so a lost connection doesn't lose data, and a compact wire format so that queue doesn't blow up memory. On the gateway side, its own forwarding queue needs to survive a gateway restart too – right now it's an in-memory structure.

AI Response Summary:

The sensor node's `main.c` was refactored (from a 400+ line monolith down to a dedicated `network.c`) to add a CoAP ACK listener, so a packet is only considered delivered once the gateway actually confirms it, and an NVS-backed store-and-forward queue so unacknowledged readings persist across radio drops. The wire format was then migrated from JSON to CBOR specifically to make that persistent queue viable – CBOR's binary encoding is small enough to hold meaningfully more backlog in the same flash budget than JSON would allow. On the gateway side, the forwarding queue was migrated from SQLite to Redis (`queue_manager.py`, `docker-compose.server.yml`) so queued-but-unforwarded packets survive a gateway container restart and support the active-active HA topology rather than being tied to a single SQLite file on one instance.

This NVS-based approach was later replaced by the RAM-based queue described in Part II (§10) once flash contention with OpenThread's own persistent storage was discovered – documented there as a deliberate architectural reversal, not a contradiction of this entry.

2. Historical Timestamp Reconstruction and a Sensor-Reboot Edge Case

User Prompt:

> When a sensor comes back online after being offline for a while, its queued backlog packets all land at the gateway within the same second or two, but their timestamps in the database should reflect when they

were actually sampled, not when they arrived. Have the gateway reconstruct historical timestamps for backlog packets using the sensor's uptime counter as an offset from the newest known reading.

AI Response Summary:

`queue_manager.py`'s `enqueue()` was updated to track each node's most recently seen `uptimeMs` in Redis. When an incoming packet's `uptimeMs` is lower than the last known value, it's recognized as a backlog packet, and its real timestamp is reconstructed by subtracting the uptime offset from the current time rather than stamping it with arrival time.

A follow-up correction was necessary once this ran against a real physical reboot: if a sensor's MCU actually reboots, its uptime counter resets to near-zero, and the original logic misread that as an enormous backlog offset (current uptime far below the last known maximum), reconstructing wildly wrong historical timestamps rather than recognizing a fresh device. The fix adds explicit reboot detection – a low absolute uptime combined with a large apparent offset is treated as a genuine reboot and resets the uptime baseline for that node, rather than being fed through the backlog-reconstruction math.

3. Alert Deduplication to Prevent Unbounded Database Growth

User Prompt:

> If a sensor sits just above a threshold, the alert evaluation fires on every single incoming reading and the alerts collection grows without bound – the same logical alert gets re-inserted every cycle instead of being recognized as still-active. Add real deduplication so a persisting condition updates one alert record instead of creating a new one each time.

AI Response Summary:

`AlertService.java` was updated to check for an existing unresolved alert of the same type/node before inserting a new one – if one is already active, the incoming evaluation updates its last-seen timestamp instead of creating a duplicate row. `AlertRepository` gained a query to support that existence check efficiently, and `AlertController` was adjusted to match. This is what makes the later query-pattern audit's read-heavy alert queries meaningful – without deduplication, `AlertRepository`'s indexes would be fighting an unbounded, mostly-redundant collection rather than a bounded set of genuinely distinct alerts.

4. Hardware/Simulation Dual-Mode Sensor Drivers

User Prompt:

> I need to keep developing and testing the firmware and pipeline without always having the physical SCD41/CCS811/SPS30 sensors wired up, but the report and defense need to be clear about which results came from real hardware versus simulated data. Add a build-time switch between real sensor drivers and a simulation mode, driven by actual datasheet-realistic value ranges rather than arbitrary random numbers.

AI Response Summary:

Physical I2C drivers for SCD41, CCS811, and SPS30 were integrated (`app.overlay`, `Kconfig`, `prj_hw.conf`), with a separate Kconfig-gated simulation path that generates values within each sensor's real datasheet operating range rather than arbitrary randoms, so simulated readings still exercise the same alert thresholds and ML feature ranges meaningfully. The build mode is a compile-time flag (`prj.conf` vs `prj_hw.conf`), not a runtime toggle – a given firmware image is unambiguously either hardware or simulation, which matters for being able to state plainly which of the report's results came from physical devices.

5. Active-Active High-Availability Load Balancing

User Prompt:

> I want the platform tier to survive losing either backend replica without operator intervention. Nginx is currently just round-robining HTTP, but the gateway's CoAP ingestion path is UDP and doesn't get the same failover story. Set up an active-active HA layer – Nginx handling both UDP and HTTP load balancing across two API instances – and give me a load-testing script that can actually prove failover works under concurrent traffic, not just in theory.

AI Response Summary:

`docker-compose.ha.yml` adds a second API replica behind Nginx. Because the gateway's CoAP ingestion is UDP, not HTTP, it needed its own load-balancing path – `nginx-udp.conf` configures Nginx's `stream` module to distribute UDP traffic across both replicas, separately from `nginx-orchestrated.conf`'s HTTP upstream block. `platform/gateway/load_test.py` was added to generate concurrent synthetic traffic against both paths and confirm neither drops requests when one replica is killed mid-load. This is the same load-balancing layer later verified end-to-end under real chaos conditions (Part III, Chaos Engineering and Reliability Test-Suite Design).

6. Digital-Twin Identity Linking (Mesh ID to Hardware ID)

User Prompt:

> The dashboard identifies nodes by their mesh-layer address (RLOC16/ext address), which changes every time a node re-joins the Thread mesh – so a node's history doesn't visually persist across reconnects. I want a stable hardware identity that survives mesh churn: link each node's actual hardware ID to whatever mesh address it currently holds, end to end from firmware through the gateway and backend, so the UI always shows "this is physical node X" regardless of its current mesh address.

AI Response Summary:

`network.c` was updated to include the sensor's hardware ID in its telemetry payload alongside the mesh-layer address. `platform/gateway/coap/server.py` and `telemetry/model.py` pass the hardware ID through unmodified rather than deriving identity solely from the mesh address. On

the backend, `SensorNode.java`, `NodeResponse.java`, and `SensorDataService.java` were updated to key node identity on hardware ID while still tracking the current mesh address as a mutable attribute of that entity. A follow-up UI change then mapped live mesh nodes onto this stable identity, so node cards and history persist across a physical device's re-joins instead of appearing as a new, unrelated node each time.

Part II—Coordinated Multi-Tier Feature Engineering

7. Pre-Inference Sanity Filter Against Hardware-Fault Poisoning

User Prompt:

> When a sensor has a hardware fault it reports a sentinel placeholder value (-999.0) rather than a real reading, but the ML poller was still feeding those straight into inference. That means a single faulty sensor can poison the anomaly detector's feature vector and the model with garbage. Add a check before inference that recognizes these sentinel values and skips the ML step entirely rather than scoring or learning from them.

AI Response Summary:

`poller.py` gained a sanity check immediately before the feature vector is built: if any channel's value is at or below -900.0 (the sentinel fault range), inference is skipped for that cycle and a warning is logged, rather than letting the fault value flow into the IsolationForest as a legitimate extreme reading. This is a narrow, targeted fix – it does not touch how sentinel values are handled elsewhere (e.g. the backend's own hardware-fault alert path, Part III §15), it only prevents them from being treated as real data by the ML model.

8. Statistical Feature-Vector Expansion (10 to 37 Dimensions)

User Prompt:

> The ML service only sees instantaneous point readings, which throws away transient spikes between polls – a VOC or CO2 event lasting 10 seconds inside a 30-second sampling cycle is invisible to it. I want the firmware to maintain rolling statistics (mean/std/max) per sensor over a circular buffer, transmit them alongside the instantaneous reading, and have the whole pipeline – gateway CBOR parsing, backend model/DTOs, and the ML feature vector – carry that through end to end.

AI Response Summary:

Firmware: a `sensor_stats` structure with a 30-sample circular buffer was added, with `stats_add_sample()`/`stats_calculate()` computing mean/std/max per channel across all three sensor polling threads (SCD41, CCS811, SPS30). This expanded the CBOR payload from 16 to 43 fields (10 instantaneous values + 27 statistics). Gateway parsing (CBOR field decoding) and the backend `SensorData` model/DTOs were updated to carry the additional 27 fields through to storage. On the ML side, the feature vector expanded from 10 to 37 dimensions – 9 sensor channels $\times$ (mean, std, max) – so IsolationForest trains on the shape of recent behavior rather than a single instantaneous point. This is also the point at which `docs/final/ml-service.md`'s dimensionality claims began drifting from the implementation, which the later documentation audit (Part III) caught.

9. Phase 2 – Batch Transmission and Sampling-Rate Decoupling

User Prompt:

> Right now the sensor node samples and transmits on the same 30-second cycle, so sampling granularity is capped by how often we're willing to key the radio. I want to decouple those: sample every 6 seconds for finer granularity, but keep transmitting in batches every 30 seconds so we're not burning radio airtime 5x more often. It has to stay backward-compatible with the existing single-sample packet format on the gateway.

AI Response Summary:

Firmware now buffers 5 timestamped samples between transmissions; a new `network_send_coap_batch()` CBOR-encodes them as `[[samples_array],{statistics_map}]`. CoAP buffers were widened to 1024 bytes to fit the batch, and `zcbor_tstr_put_term()` was required for encoder compatibility with this structure – the original encoding calls silently produced malformed output at this payload size. On the gateway, batch vs. single-sample format is auto-detected on receipt so existing Phase 1 packets still parse correctly, and batches are split into individual samples before timestamp reconstruction, working backward from a precomputed max-uptime bound so per-sample timestamps stay accurate within the batch window. Verified on physical nRF52840 hardware: the dashboard shows genuine 6-second intervals while radio transmissions remain 5x less frequent than sampling.

10. Phase 3.1 – RAM-Based Offline Storage and Store-and-Forward

User Prompt:

> The sensor node needs to survive a gateway or OTBR outage without losing data – buffer readings locally and flush them once connectivity comes back, store-and-forward style. Don't use flash/NVS for this; that's already ruled out because it conflicts with OpenThread's own persistent dataset storage on the same device. Give me a real coverage estimate in minutes, not just "it buffers." Also, if delta/differential compression on the CBOR payloads would meaningfully extend that coverage, evaluate it – but only keep it if it actually helps.

AI Response Summary:

A 50-packet circular buffer was implemented entirely in RAM (`storage.c`), avoiding flash specifically because Zephyr's flash-backed settings subsystem was already in contention with OpenThread's own persistent dataset storage on

the same device. At 1024 bytes/packet that's 50 KB of RAM for roughly 25 minutes of offline coverage at the 30-second batch interval. Gateway connectivity is tracked with a timeout (60s, 3 consecutive failures) so the sensor knows to queue rather than discard on a failed send.

Delta/byte-level compression on the CBOR batches was implemented and measured, not assumed: it produced **101.2% overhead** (467 to 473 bytes per batch), because CBOR's already-compact binary encoding doesn't have the redundant structure delta compression is designed to exploit. That result is documented as a negative finding, and the compression path was reverted in favor of the plain RAM queue rather than kept for the sake of having shipped it.

11. Fixing a Gateway-Recovery Deadlock in the Offline Queue

User Prompt:

> The offline queue fills up correctly when the gateway goes down, but it never drains once the gateway comes back – the sensor just keeps queueing forever even with a live gateway sitting right there. Find out why recovery isn't being detected.

AI Response Summary:

The sensor was gating every transmission attempt behind a `gateway_reachable` flag that, once set false, had no code path that ever re-attempted a real send to check whether the gateway was back – so the flag could never flip back to true on its own. The fix removes that pre-check entirely: the sensor now always attempts transmission regardless of the last known state, so a live gateway is detected the moment it actually accepts a packet again, and the queue drains automatically from that point rather than requiring a manual restart.

12. ML Algorithmic Hardening: Robust Scaling, Drift Detection, and R2-Gated Forecasting

User Prompt:

> Warm-up baselining and forecasting have both been naive so far. The anomaly detector needs real drift handling for a system meant to run for weeks unattended, and the forecaster needs a way to know when its own regression fit is unreliable instead of silently emitting a bad prediction. Add robust (median/IQR-based) scaling in place of raw z-score normalization, a drift-detection mechanism that can recalibrate the baseline over time, and gate the forecaster's output behind its own R^2 so it doesn't report a forecast it doesn't actually trust.

AI Response Summary:

`ml_engine.py` was reworked to scale features using median/IQR rather than raw mean/standard deviation, which is far less sensitive to the outliers anomaly detection is specifically trying to catch. A drift-detection path was added that periodically re-evaluates whether the room's baseline has genuinely shifted (occupancy pattern change, seasonal drift) and triggers recalibration rather than treating the original training baseline as permanent. `forecaster.py` was updated to compute its own R^2 against recent data and suppress low-confidence forecasts rather than surfacing an unreliable linear projection as an alert.

Documented here for continuity: this is the same drift-detection/recalibration subsystem that a later end-to-end evaluation (Part III, ML Anomaly Detector) found had several concrete instability bugs. It shipped as designed at this stage; the instability was only caught once the detector was evaluated against real multi-day recordings months later.

13. Role-Based Access Control

User Prompt:

> The dashboard is currently wide open – anyone with the URL has full access to alerts, node control, and settings. Add real authentication: a login screen, an auth context that gates routes, and at least a basic role distinction between an operator who can view/acknowledge alerts and an admin who can change thresholds or resolve infrastructure alerts.

AI Response Summary:

`AuthContext.jsx` was added to hold session/token state, with `App.jsx` and `DashboardLayout.jsx` gating routes behind it and redirecting unauthenticated users to a login screen. Role-conditional rendering was added across the alert views (`AllAlertsTable`, `OperationalTable`, `InfraTable`, `AiInsightsTable`, `MobileAlertCard`) so destructive actions (resolve, bulk-resolve) are hidden from lower-privilege roles. Caveat carried forward honestly: this is UI-level gating. Whether the backend API also rejects a lower-privileged token attempting the same action server-side was not verified as part of this change and should be checked separately before treating it as a security boundary rather than a UX one.

14. Self-Healing Alarm Resolution and Standardized Warm-Up Window

User Prompt:

> The baseline warm-up length is inconsistent between the ML service config and what the dashboard tells the operator, and alerts that resolve themselves once conditions normalize still require a manual acknowledgment – the sensor can be back in range for an hour and the alert just sits there. Standardize the warm-up window to one real number across both tiers, and make transient/threshold alerts auto-resolve once the underlying condition has been confirmed gone.

AI Response Summary:

The warm-up baseline was standardized to 40 samples in both `AlertService.java` and `app/core/config.py`, which had previously drifted out of sync. Self-healing resolution logic was added so threshold and transient alerts clear automatically once their triggering condition has been absent for a confirmation period, instead of persisting until an operator manually dismisses them. This 40-sample default is also what makes the warm-up window dashboard-configurable per deployment rather than a hardcoded constant.

Part III—Verification, Debugging, and Test-Suite Engineering

15. Multi-Engine Alert Architecture Audit

User Prompt:

> Audit `docs/final/backend-tier.md` against the actual alert-generation code in `platform/backend`. My understanding is that alerts are produced by more than one mechanism (static thresholds, statistical/heuristic rules, hardware health checks, and externally-submitted ML alerts), but the current doc doesn't distinguish them. For each mechanism, identify:
>
> - trigger conditions and input variables
> - whether it's computed in the backend or consumes precomputed features
> - persistence and deduplication behavior
> - whether the doc's description matches the code, and classify each finding as VERIFIED or INFERRED
>
> Do not assume `@Async` is in use anywhere unless you find it in the code.

AI Response Summary:

The backend implements four distinct alert-generation paths, not one unified "alert engine":

1) **Static Threshold Engine** – deterministic bound checks (`TEMPERATURE_HIGH/LOW`, CO2, PM, TVOC) evaluated synchronously against each incoming reading in `SensorDataService`. VERIFIED.
2) **Dynamic Heuristic Engine** – transient-event detection (`TRANSIENT_VOC_EVENT`, `TRANSIENT_CO2_EVENT`, `PARTICULATE_EVENT`, `COMBUSTION_WARNING`, etc.) using rolling mean/std/max and adaptive thresholds. The backend **consumes** these statistical features rather than computing the rolling window itself – the window calculation lives upstream. VERIFIED for the trigger logic; INFERRED for where the rolling statistics are actually produced, since that code is outside this repository's backend module.
3) **Hardware and Topology Health Engine** – sensor/connectivity diagnostics (`HARDWARE_FAULT`, `NODE_OFFLINE`), including sentinel-value handling (`-999.0`) and timeout-based offline detection with automatic resolution. VERIFIED.
4) **External ML Engine** – anomaly and forecast alerts (`ML_ANOMALY`, `FORECAST_WARNING`) submitted via `AlertController/CreateAlertRequest` from the ML service, not computed in the backend at all. VERIFIED.

One correction to the existing doc: it described alert evaluation as asynchronous (`@Async`). No such annotation exists on the evaluation path – it runs synchronously on the ingestion thread. This was corrected in `docs/final/backend-tier.md`, along with the addition of an Alert Lifecycle section (ingestion to evaluation to deduplication/debounce to categorization to severity assignment to resolution to archival) and an updated feature-classification summary distinguishing Active/Inferred/Unsupported behavior.

16. ML Anomaly Detector: Diagnosing Near-Zero Detection Accuracy

User Prompt:

> I ran `evaluate.py` against a 2.8-day recording with injected hazard events and the detector is barely catching anything while also throwing constant false positives. Before I write this up as a limitation in the report, I want to know if this is a genuine limitation of Isolation Forest on this feature set, or a bug in the evaluation/detection pipeline. Walk through `ml_engine.py`, `forecaster.py`, and `evaluate.py` and tell me which it is.

AI Response Summary:

It is a pipeline bug, not a modeling limitation – seven distinct issues were compounding:

- `evaluate.py` was using the training-set length as the drift-detection window instead of the production default, which suppressed detection almost entirely.
- The IQR-floor safeguard (intended to stabilize flatlined sensors) was also collapsing the scale of legitimate near-zero-variance rate-of-change features, turning small real deviations into apparent extreme outliers.
- Drift-triggered retraining had no safeguard against absorbing the anomaly itself into the new "normal" baseline.
- Sentinel fault values (`-999`) were leaking into rate-of-change calculations and producing fake spikes.
- Refitting the entire IsolationForest ensemble on small recalibration buffers was inherently unstable regardless of buffer size; recalibration was changed to update only the median/IQR scaling center, keeping the original trained forest fixed.
- Recalibration was firing far more often than genuine environmental drift warrants – a cooldown was added.
- The evaluation harness's injected test anomalies were sized below this room's own ordinary variability, so they weren't testing a genuine hazard; they were rescaled against ASHRAE/WELL v2 safety thresholds, and a consecutive-reading confirmation requirement was added to suppress transient-noise false positives.

After these fixes, re-running the same 2.8-day recording moved detector performance from **TPR 8.95% / FPR 57.55%** to **TPR 77.31% / FPR 0.45%**. This is now documented as a measured result in the evaluation section rather than a stated limitation, with the before/after numbers and root causes retained in the commit history for traceability.

17. Multi-Event Detection-Latency Distribution for the Presentation

User Prompt:

> The evaluation so far only injects a single CO2 hazard event and reports one detection-latency number,

which isn't a distribution – it's an anecdote. For the presentation's statistics section I need a real distribution: inject several independent CO2 events across the recording and report mean, standard deviation, and a confidence interval on detection latency, not just one sample.

AI Response Summary:

`evaluate_latency_distribution.py` extends the `evaluate.py` methodology from the previous entry to inject 6 separate CO2 hazard events spread across the test recording instead of one, measuring detection latency independently for each and reporting the resulting mean/std/CI rather than a single anecdotal figure. This deliberately runs against the already-corrected detector – the distribution is only meaningful because the detector's false-positive rate was brought under control first; running this evaluation before that fix would have produced a latency distribution contaminated by spurious early triggers.

18. Persistence-Layer Query-Pattern Audit

User Prompt:

Produce a read-only audit of every repository method in `platform/backend` – no file modifications. For each method: collection, filter fields, sort fields, pagination, expected complexity, and index utilization. Separately identify read-heavy vs. write-heavy paths, aggregation pipeline usage, and any bulk/transactional operations. Classify every finding as VERIFIED, INFERRED, or NOT FOUND, and flag hot paths or scaling risks at the end.

AI Response Summary:

(Full report on file at the time; representative excerpt below.)

SensorDataRepository – `findTop400ByNodeIdOrderByTimestampDesc`, `findAllByOrderByTimestampDesc(Pageable)`, `findByNodeIdAndTimestampBetween`, etc. – all VERIFIED against a compound `nodeId+timestamp` index, O(log N) lookups, consistent use of bounded result sets (Top10/Top400) or explicit pagination rather than unbounded scans.

AlertRepository – filter/sort patterns for `resolved`, `type`, and `severity` fields, all index-backed. No unbounded queries found.

No transactions, optimistic locking, or bulk write operations were found anywhere in the repository layer (NOT FOUND) – all writes are single-document upserts. The main scaling risk identified was the absence of a TTL or archival policy on `sensor_data`, meaning index size grows unbounded with retention; this was flagged as a recommendation rather than acted on, since changing retention policy was out of scope for the audit.

19. ML Service Documentation-vs-Implementation Audit

User Prompt:

Compare `docs/final/ml-service.md` section by section against `app/main.py`, `app/services/poller.py`, `app/ml/ml_engine.py`, and `app/ml/forecaster.py`. Check formulas, constants, thresholds, endpoints, hyperparameters, and file paths. Classify each section CORRECT / PARTIALLY CORRECT / INCORRECT and list every discrepancy. Read-only – do not modify anything.

AI Response Summary:

Most of the document matched the implementation. Two discrepancies were significant enough to flag:

- **Directory structure**: the doc listed `main.py`, `poller.py`, `ml_engine.py`, `forecaster.py` as flat files; the actual layout nests them under `app/`, `app/services/`, and `app/ml/`.
- **Feature vector dimensionality mismatch**: the background poller in `app/services/poller.py` correctly builds a 12-dimensional feature vector (5 raw values, 5 rates-of-change, 2 cyclical time encodings), but the synchronous `/analyze` endpoint in `app/main.py` was building a 10-dimensional vector of raw values only, omitting the rate-of-change and cyclical features entirely. Calling `/analyze` against a model trained by the poller would raise a numpy shape-mismatch error. This was flagged as a functional bug, not just a documentation gap.

The doc's math (Isolation Forest rationale, warm-up phase, statistical feature derivations) was otherwise CORRECT.

20. Fog-Tier Failover: Diagnosing an Asymmetric Border-Router Bug

User Prompt:

During a live failover test – unplugging the primary OpenThread dongle – the Mac fog node correctly took over as mesh leader, but the dashboard showed every sensor node as offline and the gateway logs showed no incoming traffic at all, even though the mesh-layer failover itself worked. The Linux fog node doesn't have this problem. Compare `mac_startup.sh` against `linux_startup.sh` and find the divergence.

AI Response Summary:

The mesh-layer failover was working correctly – the bug was one layer up, in how the Mac fog node bridges Thread traffic to the gateway. `mac_startup.sh` starts its `socat` UDP bridge before the `wpan0` Thread interface has finished initializing, and unlike `linux_startup.sh`, it never explicitly binds `socat` to the shared anycast address (`fd5e:a726:ec55:ee12::1`). As a result, sensor CoAP packets sent to that anycast address after failover were never picked up by `socat` on the Mac node, so the Python gateway received nothing – consistent with the empty gateway logs. Because `NodeHealthMonitor` on the backend marks nodes offline after a fixed no-data timeout, the dashboard correctly (if confusingly) reported the sensors as offline, even though the sensors themselves were transmitting normally.

The fix ports the explicit `socat` bind logic from `linux_startup.sh` to `mac_startup.sh` and reorders startup so

the bridge attaches only after the Thread interface is confirmed up.

21. Historical Topology Scrubber: Link-Quality Integration and a Rules-of-Hooks Fix

User Prompt:

> `TopologyScrubber.jsx` needs to plot event markers (node join/leave, leader change, gateway status change) on the history slider, derived from the `historyTimeline` array, and I want it colour-coded by event type with hover detail. Whatever you implement has to keep the hook call order stable across renders – I don't want a Rules-of-Hooks violation creeping back in once this touches conditional slider state.

AI Response Summary:

Event-mark computation was implemented as a `useMemo` over `historyTimeline` (recomputing on data change, not on every render), replacing an earlier boolean flag (`historyTimeline.length<=20`) that only toggled whether *any* tick marks showed at all. The new derivation walks consecutive timeline entries and flags a mark when node count changes, the leader's `extAddr` changes, or `gatewayHealth.status` changes – producing an array of `{value:index}` marks consumed directly by MUI's `Slider`. Because the derivation is a plain `useMemo` computed unconditionally at the top of the component (not inside a branch or callback), hook invocation order stays identical across renders regardless of playback state.

A follow-up change added colour differentiation at the mark level – join/leave in blue, leader change in orange, gateway status in red – plus hover tooltips carrying the specific event description, and a `NodeMetricsPage` view exposing the underlying link-quality (LQI) time series per node so the marks can be cross-referenced against raw radio metrics rather than taken at face value.

22. Chaos Engineering and Reliability Test-Suite Design

User Prompt:

> The professor's stated requirement is: "run your system 10 times and see how many times it failed" as a reliability measure. I have a script that boots the full Docker Compose stack (Mongo, Redis, both API instances, ML service, gateway, dashboard) 10 times and checks each one comes up clean – that literally answers the requirement. But for a Master's-level submission I think that's too weak on its own: it only tests cold-start reliability, not fault tolerance under load. Design a second test that's actually testing the architecture's high-availability properties – the Nginx load balancer failing over between the two API replicas – under realistic concurrent sensor traffic, and make the boot-reliability script's output presentable (no raw Docker log spam).

AI Response Summary:

Two complementary scripts now cover this, addressing the literal requirement and the architectural claim separately:

- **'reliability_test.sh'** answers the stated requirement directly: it runs `dockercomposeup-d` for the full stack, polls the API health endpoint with a bounded timeout, tears the stack down, and repeats for 10 iterations, reporting a pass/fail count. Docker's own build/pull output is redirected to `/dev/null` so the run produces a compact per-iteration status line instead of unreadable log volume – the only change needed for it to be presentation-safe.
- **'chaos_stress_test.sh'** and **'stress_worker.js'** test the property the boot-reliability script cannot: while `stress_worker.js` drives concurrent synthetic sensor traffic against the stack, the chaos script kills one of the two backend API containers mid-load (`dockerkillapi-1`) and verifies that Nginx's `proxy_next_upstream` configuration transparently redirects idempotent POST traffic to `api-2` with zero dropped requests, then confirms the killed container's data was never lost from Mongo.

The two scripts are reported separately in the presentation for exactly the reason they were designed separately: the first is a literal, unembellished answer to the professor's question; the second is additional evidence of the specific high-availability design decisions in Section 3 (dual API replicas behind Nginx) actually holding up under failure, not just under steady-state load.

23. Sensor Threshold and Citation Correction

User Prompt:

> Something's off with the alert thresholds – I don't think the humidity max, and possibly the CO2/temperature noise floors, actually match the SCD4x datasheet or the ASHRAE/OSHA references cited in the report. Check `AlertService.java` and the dashboard's formula-explainer dialog against the actual Sensirion SCD4x datasheet values and fix whatever's inconsistent.

AI Response Summary:

The humidity maximum threshold and the CO2/temperature/humidity noise-floor constants in `AlertService.java` did not match the SCD4x datasheet specifications, and the corresponding standard-name citations (ASHRAE/OSHA) in `AlertFormulaDialog.jsx` were inconsistent with what the report itself cites elsewhere. Both files were updated so the enforced thresholds, the dashboard's user-facing formula explanation, and the report's cited standards are now mutually consistent, rather than three independently-drifted sources of truth.

GLOSSARY

AI: Artificial Intelligence.
API: Application Programming Interface.
CBOR: Concise Binary Object Representation.
CoAP: Constrained Application Protocol.
eCO_2: Equivalent Carbon Dioxide.
FPR: False Positive Rate.
IEEE: Institute of Electrical and Electronics Engineers.

IoT: Internet of Things.
LQI: Link Quality Indicator.
ML: Machine Learning.
OTBR: OpenThread Border Router.
PM: Particulate Matter.
RCP: Radio Co-Processor.
REST: Representational State Transfer.
TPR: True Positive Rate.
TVOC: Total Volatile Organic Compounds.
WS: WebSocket.